Optimization Report

1. Object Instantiation Optimizations

- **Currency Formatter**: By migrating from `new Locale("en", "IN")` to `Locale.of("en", "IN")`, the application now leverages Java's internal caching. This prevents the JVM from allocating new memory for a Locale object every time a price is printed, significantly optimizing memory usage during large cart checkouts.

2. File I/O Efficiencies

- **Try-With-Resources**: The `FileHandler` utilizes Java's `try-with-resources` blocks (`try (BufferedReader br = new BufferedReader(...))`). This automatically manages resource cleanup and gracefully closes the file streams even if a parsing error occurs, preventing memory leaks and locked files.
- **Lazy Initialization**: The `Scanner` object in `main.java` checks for file existence and length before choosing to fall back to interactive mode. This prevents throwing unnecessary `FileNotFoundException` objects, keeping the execution flow incredibly fast.